

Clase 8: Aseguramiento de Calidad — Pruebas Web, API y Agentes con IA Generativa

Programa: Hardcore AI | 30X · **Instructor:** Andres Caicedo · **Duración:** 2h

Bienvenida

En la Clase 7 cerraste el ciclo de implementación: orquestación AI-DLC → Linear, code review automatizado y memoria de proyecto. Tu producto ya existe, está construido, revisado y documentado. Hoy entras al **filtro de calidad antes de producción**: cómo garantizar — y demostrar — que el producto se comporta como el usuario espera, en tres capas distintas (web, API y agentes inteligentes). La sesión alterna teoría con demostración en vivo sobre tres repositorios reales (`qa-e2e` , `qa-api` , `qa-agent-eval`), todos atacando un mismo producto en ejecución. Tú observas, anotas y replicas el patrón sobre tu propio producto entre clases.

De qué se trata

Ya tienes el producto implementado. Ahora la pregunta es: ¿cómo garantizas que funciona como se espera? Esta sesión cubre las estrategias y herramientas para asegurar la calidad del producto, potenciadas por IA generativa. La primera parte alterna teoría con demostraciones en vivo — desde la pirámide de pruebas hasta el patrón Persona + Juez. La segunda parte es una demostración continua del instructor sobre tres repositorios de código que prueban un producto real en ejecución: pruebas E2E web, pruebas de API, y evaluación de agentes inteligentes.

Qué vas a aprender

- Ubicar tus esfuerzos de testing en la **pirámide correcta** y reconocer cuándo estás cayendo en el anti-patrón Ice Cream Cone.
- Escribir y mantener **escenarios BDD/Gherkin** que sirven simultáneamente como spec del negocio y tests ejecutables.
- Aplicar **Page Object Model** y patrones de API testing (request builders, contract testing) para que tu suite escale sin volverse frágil.
- Implementar el **patrón Persona + Juez** con golden datasets para evaluar agentes inteligentes — algo que un test binario `pass/fail` no captura.
- Configurar el **stack de IA para QA**: Playwright + MCP + steering file, de modo que el coding agent genere tests siguiendo las convenciones de tu proyecto.

Recorrido de la sesión

1. **La Pirámide de Pruebas** — Unitarios · Integración · E2E · Performance. Dónde invertir el esfuerzo y qué evitar (anti-patrón **Ice Cream Cone**).
2. **BDD, TDD y Gherkin** — TDD como ciclo de diseño (Red → Green → Refactor), BDD como lenguaje negocio/ desarrollo, y Gherkin como spec ejecutable. Anatomía de un `.feature` file.
3. **Patrones de diseño para testing** — **POM** (encapsular UI), **Screenplay** (actores con tareas) y patrones de API (request builders, contract testing). El costo de no usar patrones.
4. **Pruebas de agentes inteligentes** — Patrón **Persona + Juez**: un LLM simula al usuario, otro evalúa la conversación contra un rubric. **Golden datasets** para calibrar al Juez.
5. **El stack de IA para testing** — Coding Agent + Playwright MCP + steering file. El agente inspecciona el DOM en vivo y genera locators precisos.

6. **Hands-on demostrativo** — `qa-e2e` (Playwright + BDD + POM), `qa-api` (endpoints + contratos + agente vía API) y `qa-agent-eval` (Persona + Juez con reporte en Linear).

 Qué vas a observar en vivo

- **Gherkin** → **step definitions** → **reporte HTML** — un `.feature` real con escenarios Given/When/Then se convierte en tests Playwright ejecutables.
- **POM con y sin patrón** — el mismo test E2E con selectores inline (frágil) vs. con clase `HomePage` (mantenible). La diferencia se ve.
- **Agent generando Page Objects desde el DOM real** — usa el Playwright MCP para inspeccionar la app en vivo y emite locators precisos, no inventados.
- **Agente Persona simulando al usuario** — el LLM recibe un perfil y genera mensajes naturales multi-turno contra el agente bajo prueba.
- **Agente Juez devolviendo un scorecard** — puntuaciones por dimensión (precisión, tono, alucinaciones) que se convierten en issues automáticos en Linear.

 Antes de la clase

- [] Tener listo tu **producto de la fase Implementación** (Clases 4-7) — corriendo en local, con scaffolding y arnés activos.
- [] **Node.js 20+** instalado (`node --version` debe responder).
- [] **IDE agéntico** configurado con MCP habilitado: Kiro, Cursor o Claude Code.
- [] Cuenta funcional en **Linear** (gratuita) — al final crearemos issues de hallazgos desde el agente Juez.
- [] Familiaridad básica con terminal y TypeScript/JavaScript.

 Tu tarea

1. Aplicar lo aprendido al propio producto: implementar al menos **3 tests E2E web y 3 tests de API** con Playwright, usando Page Object Model, generados y refinados con el coding agent.
2. Implementar el **patrón Persona + Juez** para evaluar el agente del propio producto, usando los user personas de la documentación.
3. Sin utilizar IA, escribir al menos **2 feature files con sus step definitions** automatizados, partiendo de la documentación BDD existente.
4. Configurar Playwright para generar **reportes HTML y traces on failure**.
5. Crear y activar el **steering file de testing** (`.kiro/skills/testing.md`) adaptado a las convenciones del propio proyecto.

 Qué vas a producir

Artefacto	Formato
Tests E2E web + Page Objects	<code>tests/e2e/*.spec.ts</code> · <code>pages/*.ts</code>
Tests de API (endpoints, contratos, agente)	<code>tests/api/*.spec.ts</code> · <code>tests/api/agente-api.spec.ts</code>
Evaluación de agentes (Persona + Juez)	<code>tests/agents/*.ts</code> · <code>rubrics/evaluacion.md</code> · <code>datasets/golden-dataset.json</code>
Feature files + Step definitions	<code>tests/features/*.feature</code> · <code>steps/*.ts</code>

Artefacto	Formato
Configuración Playwright + Reporte	<code>playwright.config.ts</code> · <code>playwright-report/</code>
Steering file + MCP config	<code>.kiro/skills/testing.md</code> · <code>.kiro/settings/mcp.json</code>



Herramientas que vas a usar

Herramienta	Para qué
Playwright + playwright-bdd	Framework E2E web/API + Gherkin ejecutable sobre Playwright
Playwright MCP Server	El agente navega el producto, inspecciona DOM y genera locators precisos en vivo
Coding Agent (Claude Code · Kiro · Cursor)	Genera tests, page objects, step definitions y corrige fallos usando traces
LLM Agente Persona + Juez	Dos agentes: uno simula al usuario real, el otro evalúa la conversación con rubric
Golden Datasets	Conversaciones de referencia para calibrar al Juez antes de evaluar el agente real
Steering Files / Skills	Convenciones de testing del proyecto que el agente sigue sin repetirlas en cada prompt



Repositorios de la sesión

Recomendado: clonarlos antes de la sesión para seguir los pasos en paralelo o replicarlos después.

Repo	Contenido
<code>clase8/repos/qa-e2e</code>	Playwright E2E + Page Objects + BDD/Gherkin + steering file
<code>clase8/repos/qa-api</code>	Playwright API testing + request builders + tests del agente vía API
<code>clase8/repos/qa-agent-eval</code>	Patrón Persona + Juez + golden datasets + reporte en Linear

Todos los repos prueban el mismo producto ejemplo configurado vía `BASE_URL` / `AGENT_URL`.



Recursos

- **Playwright Docs** — playwright.dev — documentación oficial, ejemplos y API reference.
- **playwright-bdd** — github.com/vitalets/playwright-bdd — Gherkin ejecutable sobre Playwright.
- **Page Object Model** — martinfowler.com/bliki/PageObject.html — el artículo de referencia de Martin Fowler.
- **Cucumber Gherkin Reference** — cucumber.io/docs/gherkin/reference — sintaxis oficial completa de Gherkin.
- **Evaluating LLM Agents (Anthropic)** — anthropic.com/research — patrones de evaluación de agentes con rubric.
- **Playwright MCP** — github.com/microsoft/playwright-mcp — el MCP server oficial de Playwright para coding agents.